Sri Lanka Institute of Information Technology

SE3090 – Software Engineering Frameworks
Year 3, Semester 1 - 2026

Practical Answer Submission

Practical Sheet No: 02

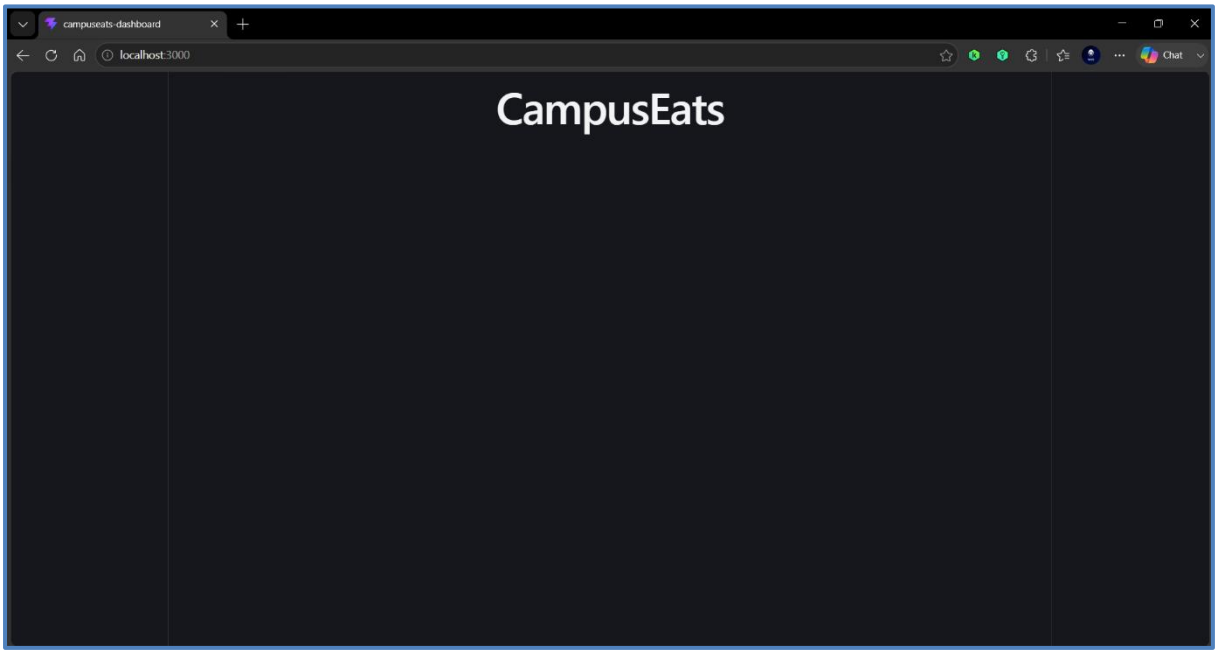
Student ID	IT24102798
Student Name	Sooriyabandara U.R.G.W.K.
Campus/ Center name	Malabe
Specialization	Artificial Intelligence
Batch No.	01.01

Task 01 – Scaffold the React App & Feature-Based Structure

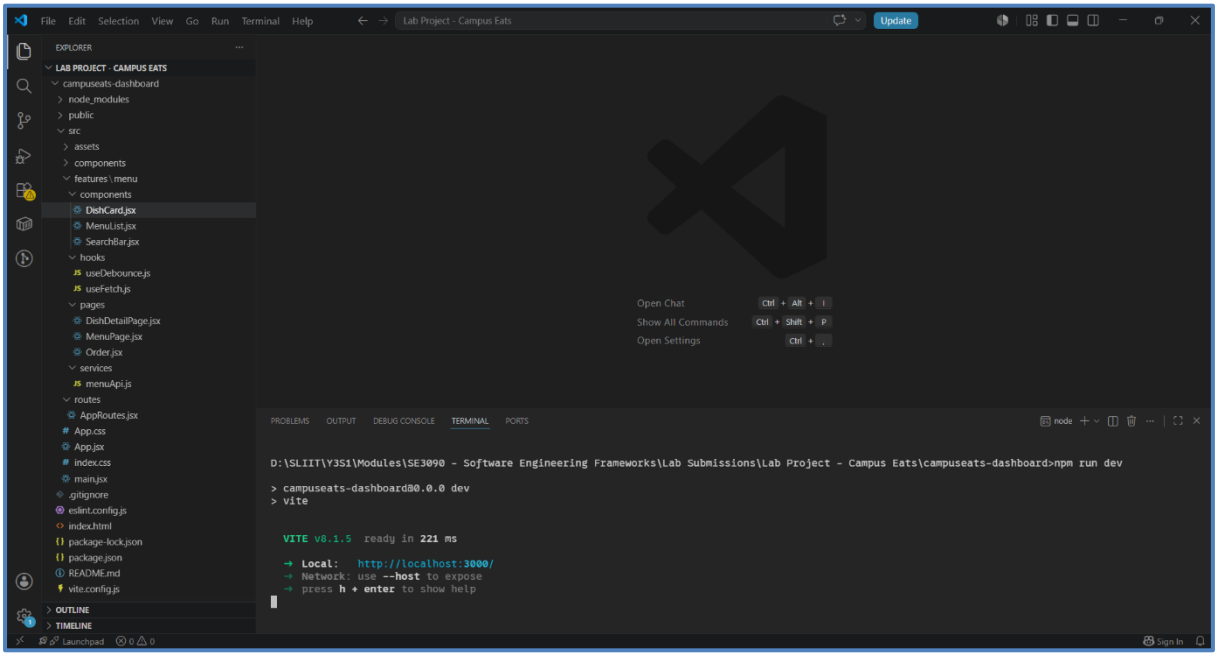
Task Explanation

- This task focused on setting up a new React application using Vite and organizing the project with a feature-based folder structure. Grouping files by feature rather than by file type improves maintainability and scalability and makes the project easier to navigate as it grows.

- **Screenshot of the Running App**



- **Screenshot of the Feature-based Folder Structure & the Vite Dev Server**

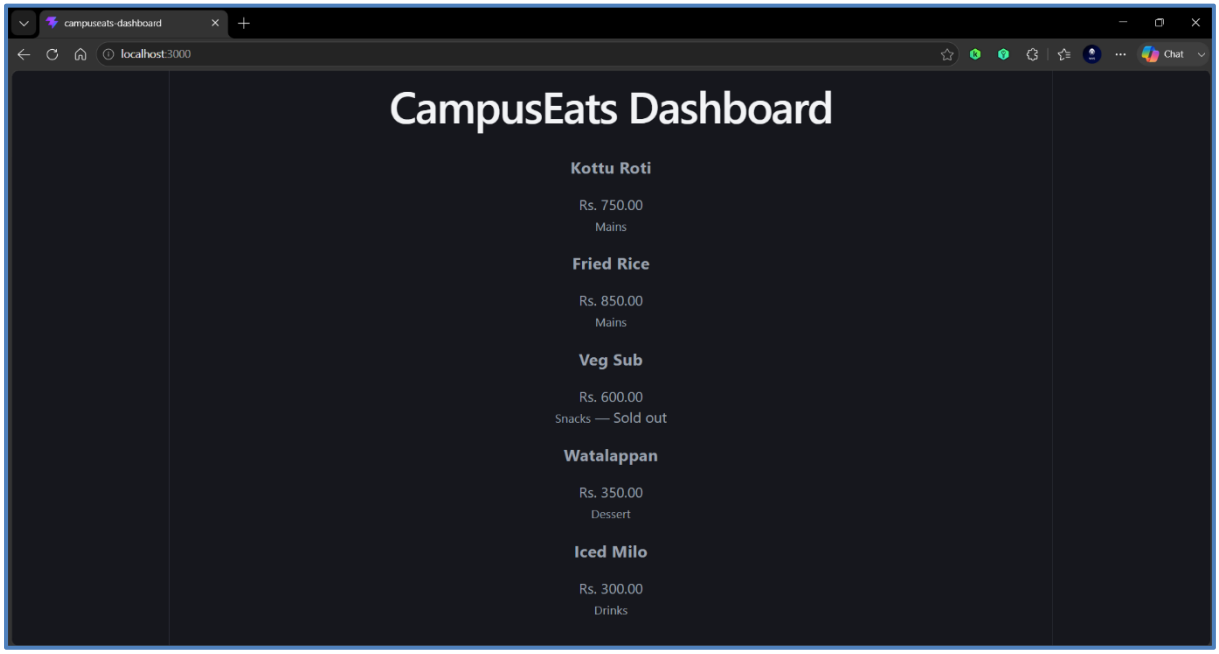


Task 02 – Reusable Components, Props & List Rendering

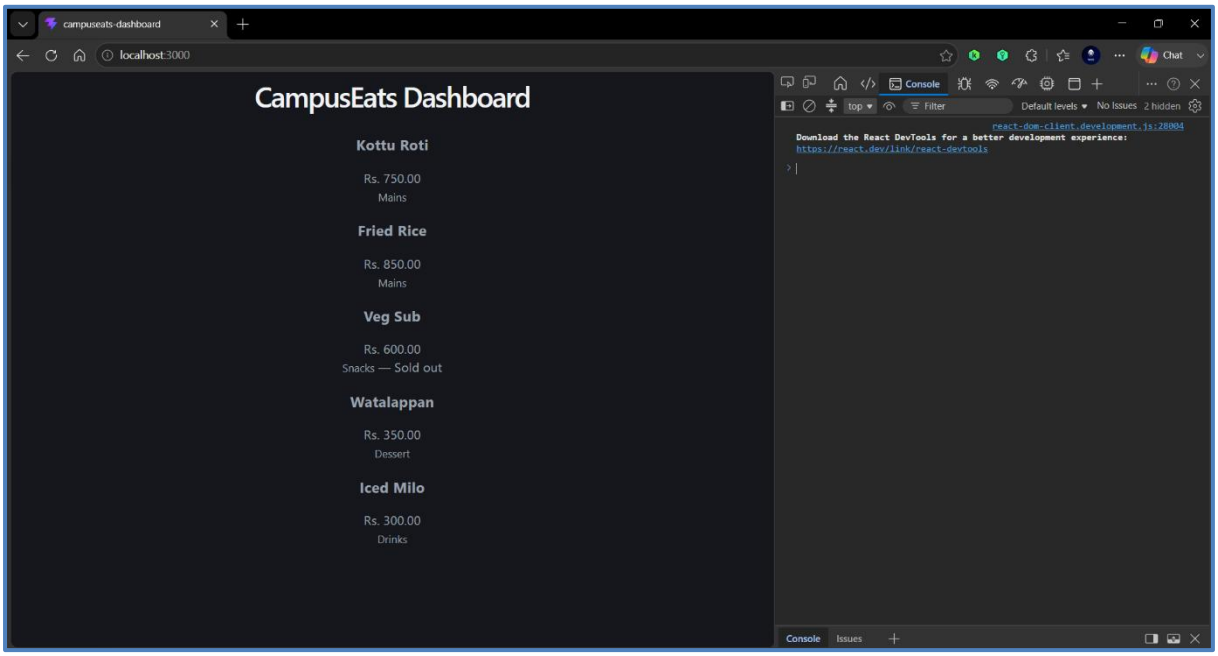
Task Explanation

- This task demonstrated component-based development by creating reusable DishCard and MenuList components. Data was passed from parent to child using props, while list rendering and conditional rendering were used to display menu items efficiently.

- **Screenshot of the list of Five Dish Cards rendered from the Array via Props**



- **Screenshot of the Console with No Missing-key Warnings**

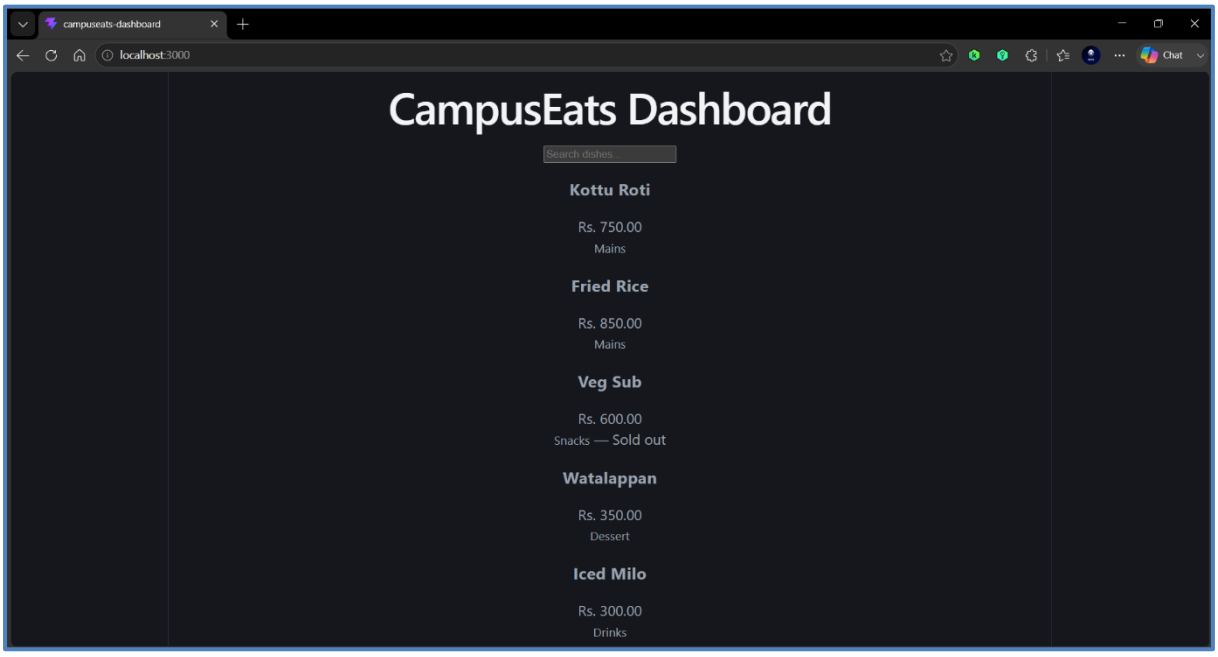


Task 03 – State, the useDebounce Hook & Event Handling

Task Explanation

- This task introduced React state management with useState and implemented a controlled search input. A custom useDebounce hook was used to delay filtering until the user stopped typing, reducing unnecessary updates and improving performance.

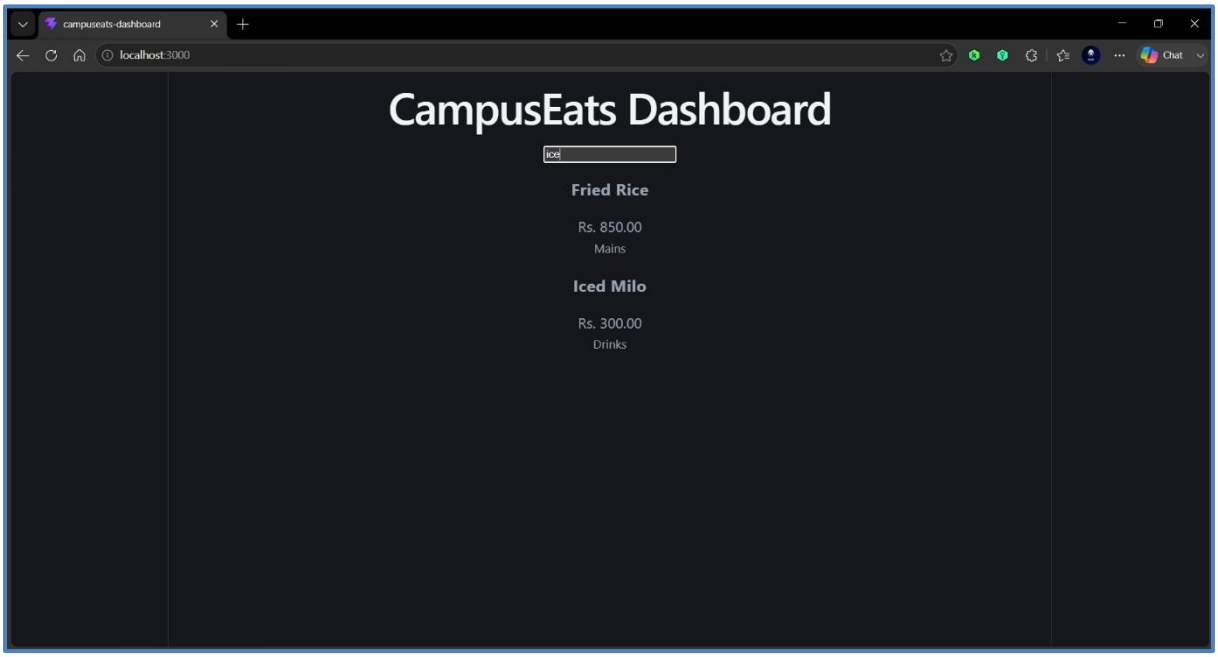
- **Screenshot of the Working Search Box that Filters the Dish List**



- **Screenshot of the Filtered Dish List – 01**

- **Screenshot of the Filtered Dish List – 02**

- **Screenshot of the Filtered Dish List – 03**

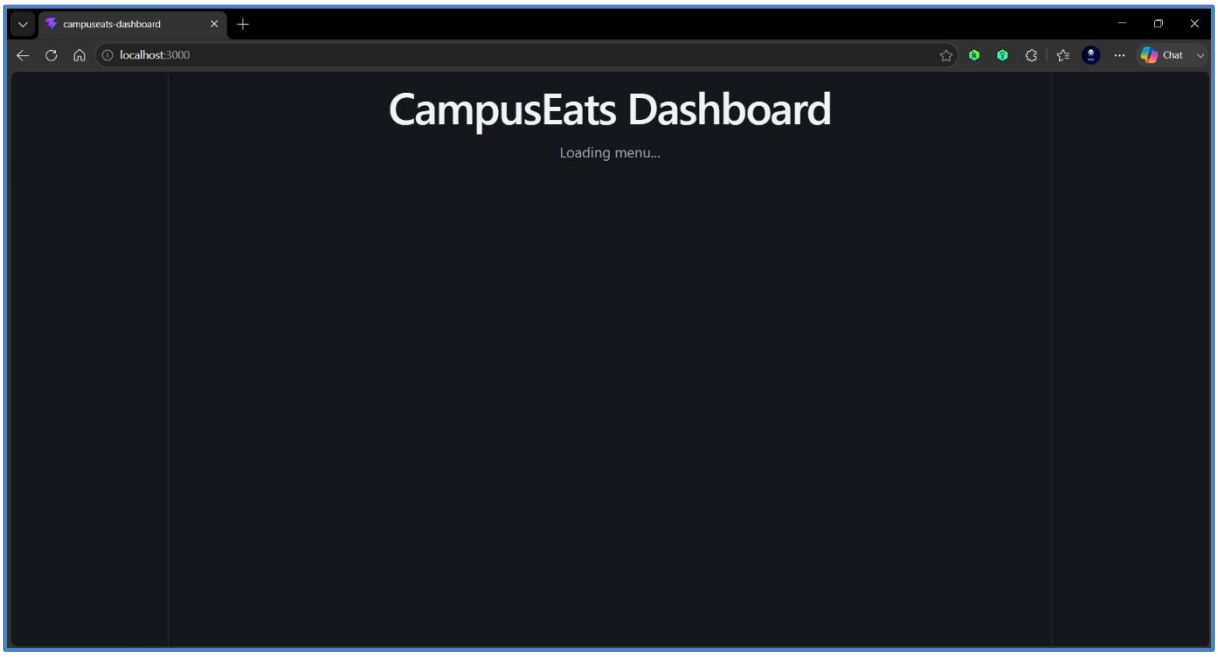


Task 04 – Data Fetching with Loading, Success & Error States

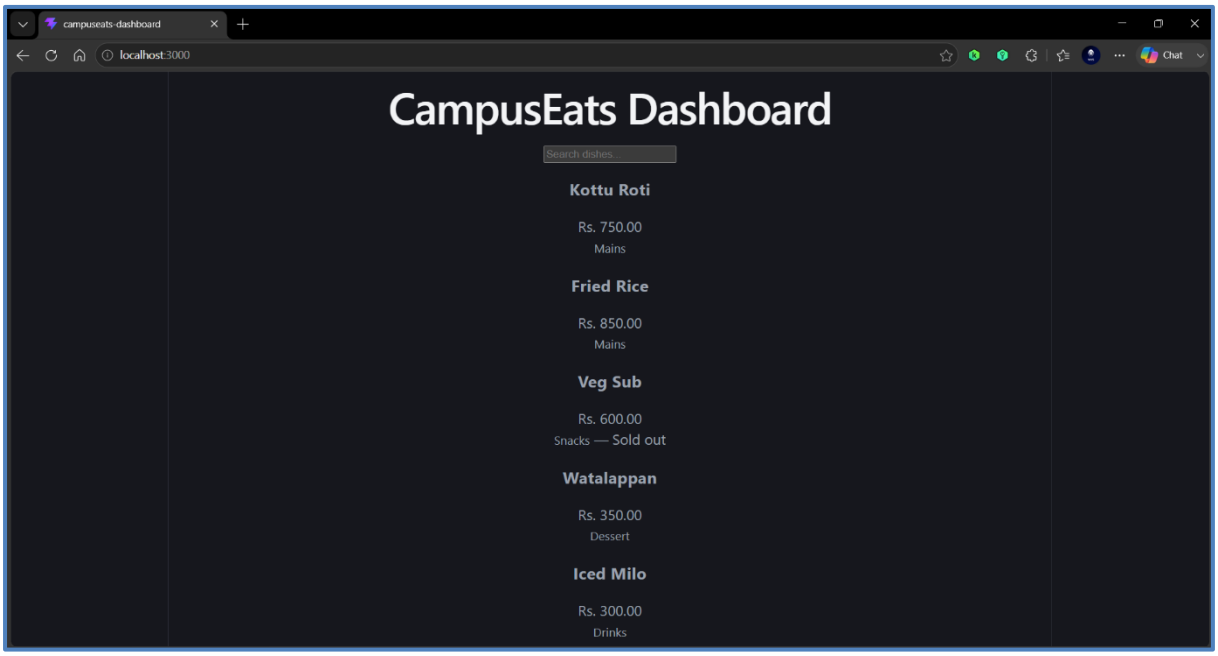
Task Explanation

- This task replaced hard-coded data with data fetched from menu.json using a reusable useFetch hook. The application handled the three essential fetch states which are loading, success, and error—to provide a reliable and user-friendly experience.

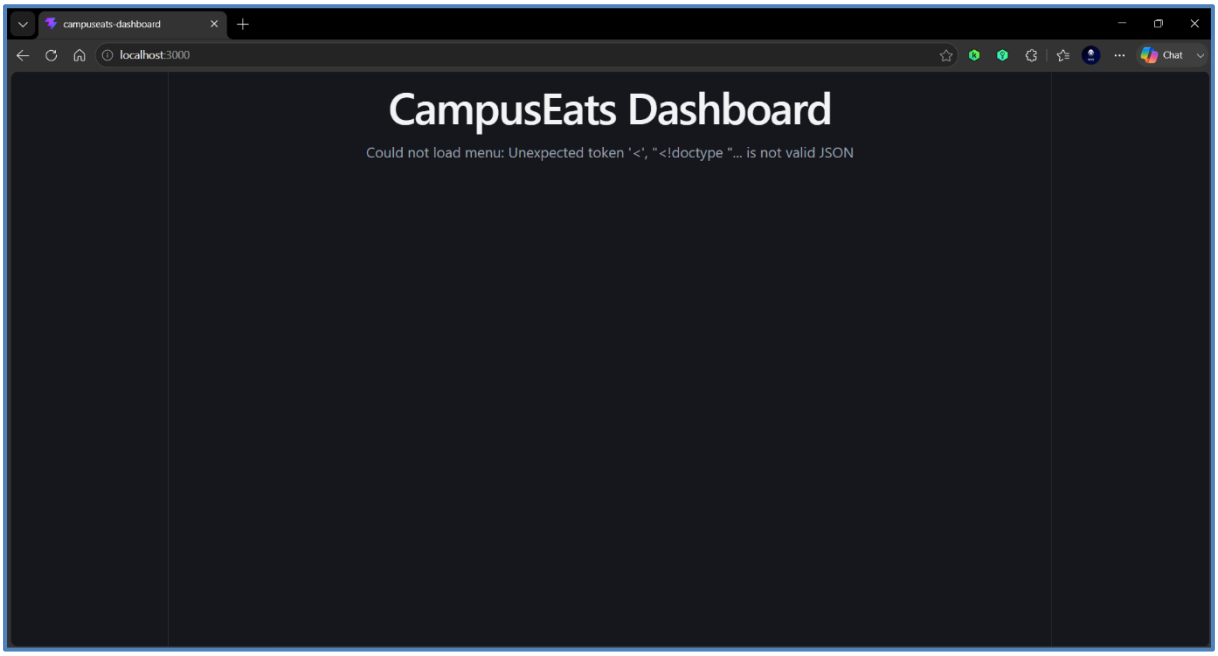
- **Screenshot of the Loading Page**



- **Screenshot of the Successfully Loaded Page**



- **Screenshot of the Error of the Wrongly Fetched JSON**



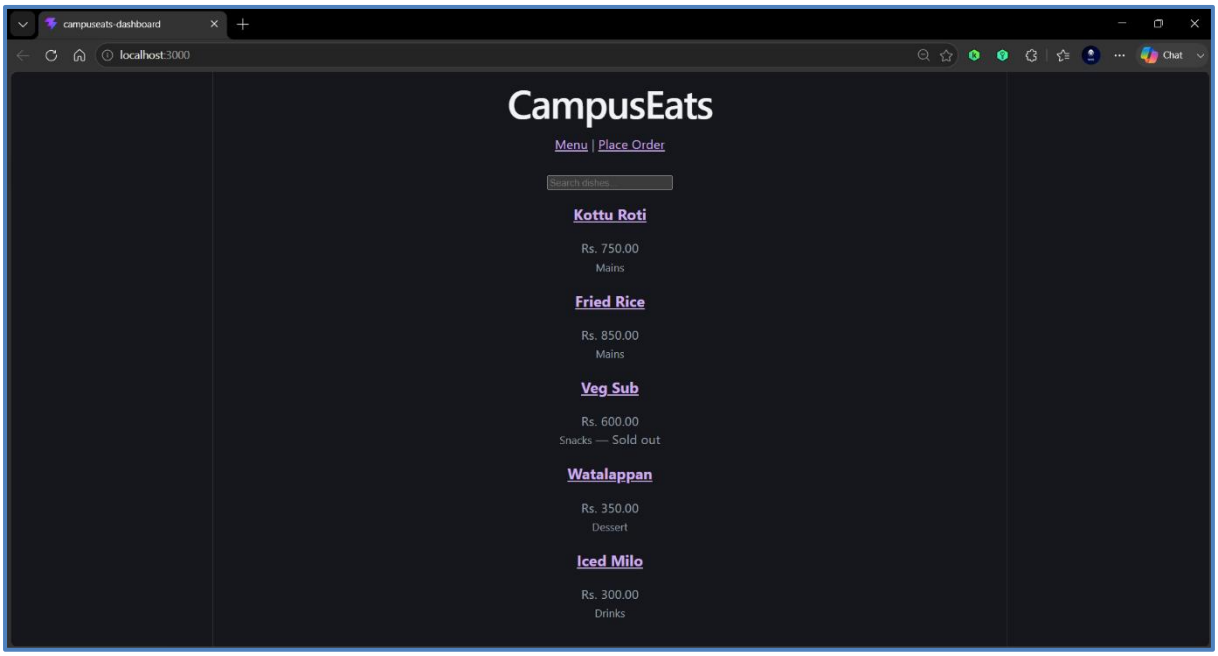
Task 05 – Routing, Navigation & a Validated Form

Task Explanation

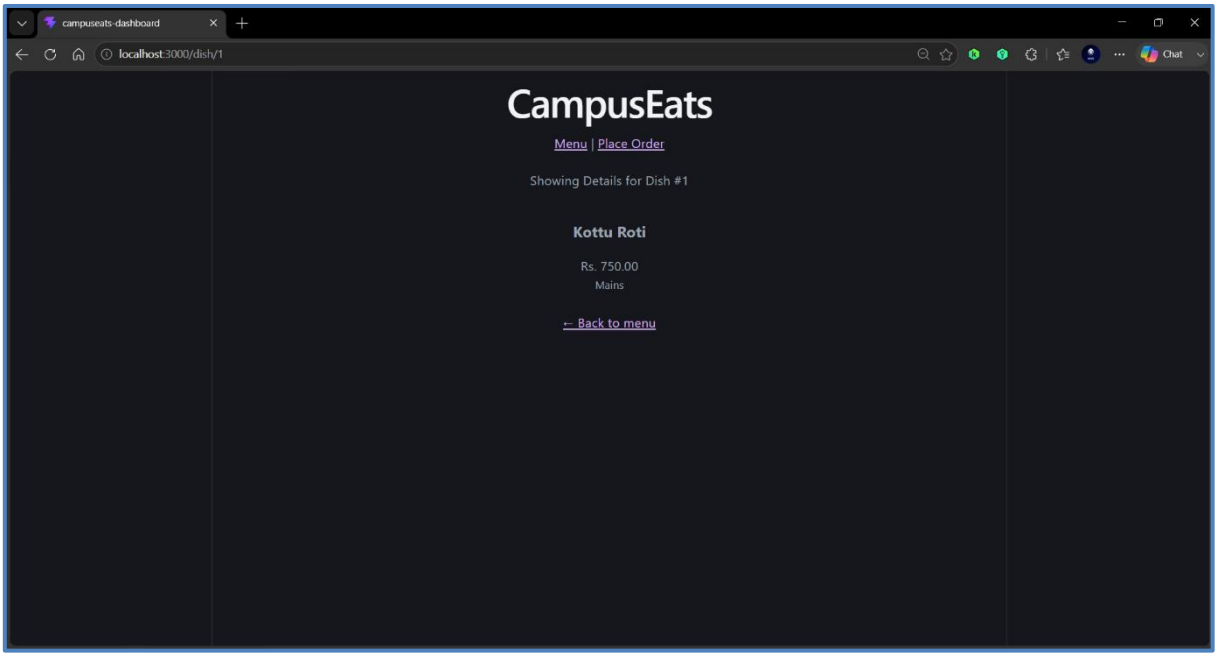
- This task implemented client-side routing with React Router, allowing navigation between the menu, dish details, and order pages while handling invalid routes with a 404 page. It also included form validation using React state to prevent invalid submissions and provide feedback to the user.

Part A – Routing & Navigation

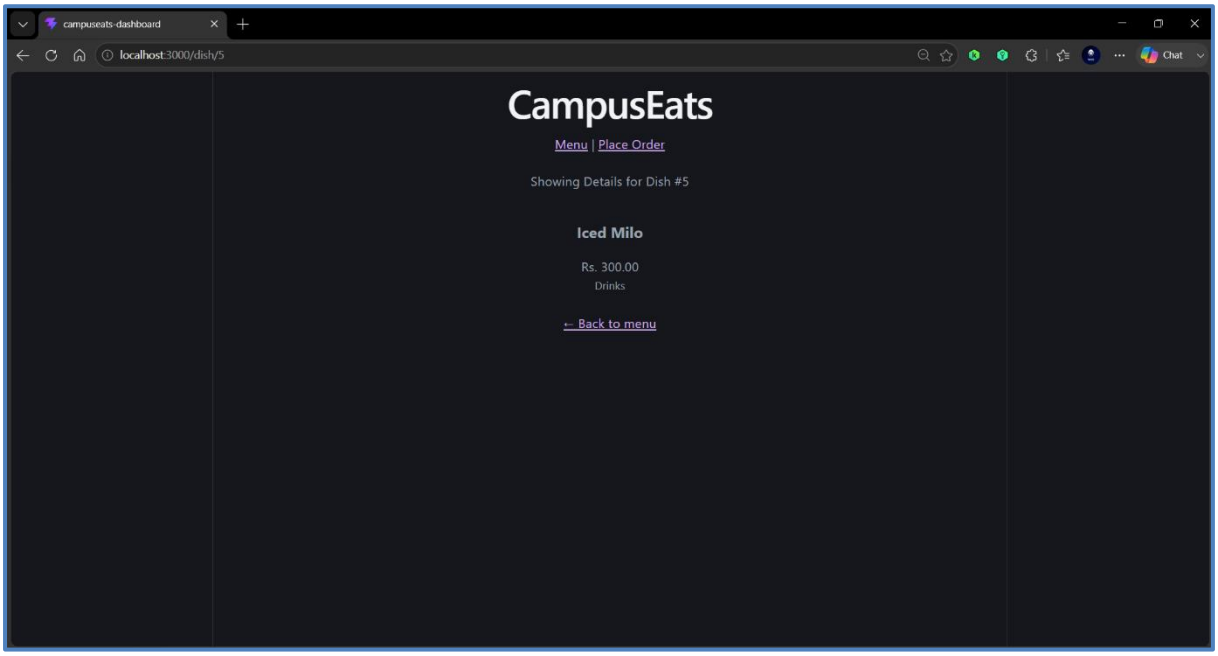
- **Screenshot of the Finalized Routing and Navigation**



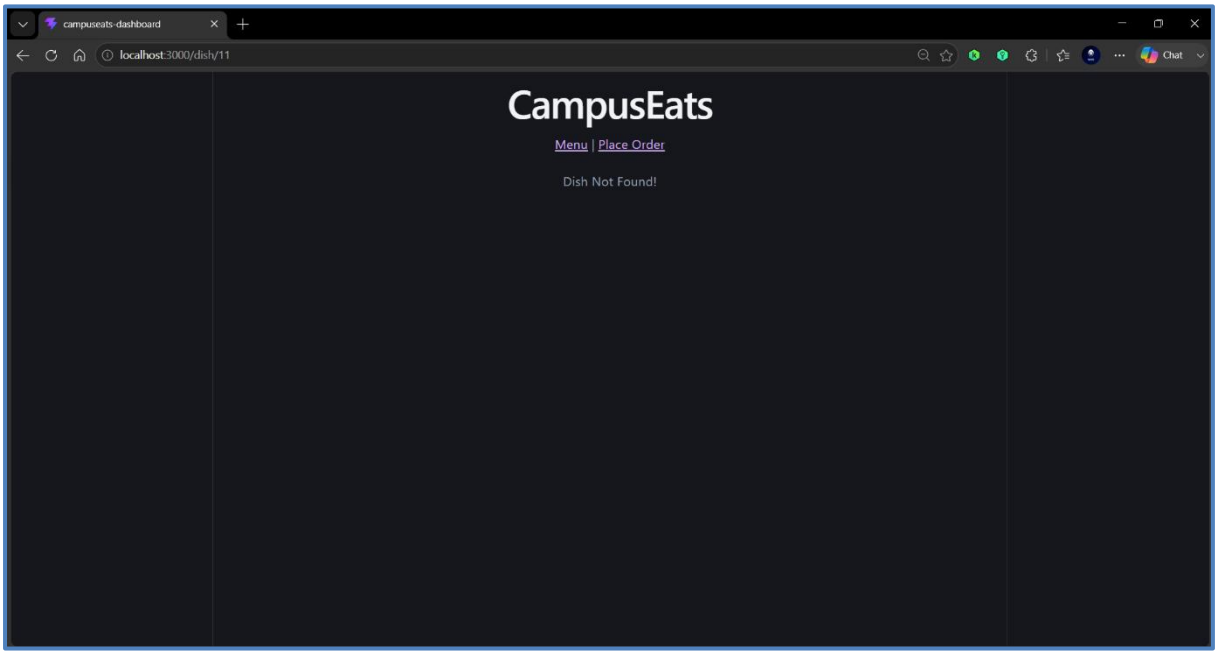
- **Screenshot of the Routing of Detailed Pages of Each Dish – 01**



- **Screenshot of the Routing of Detailed Pages of Each Dish – 02**

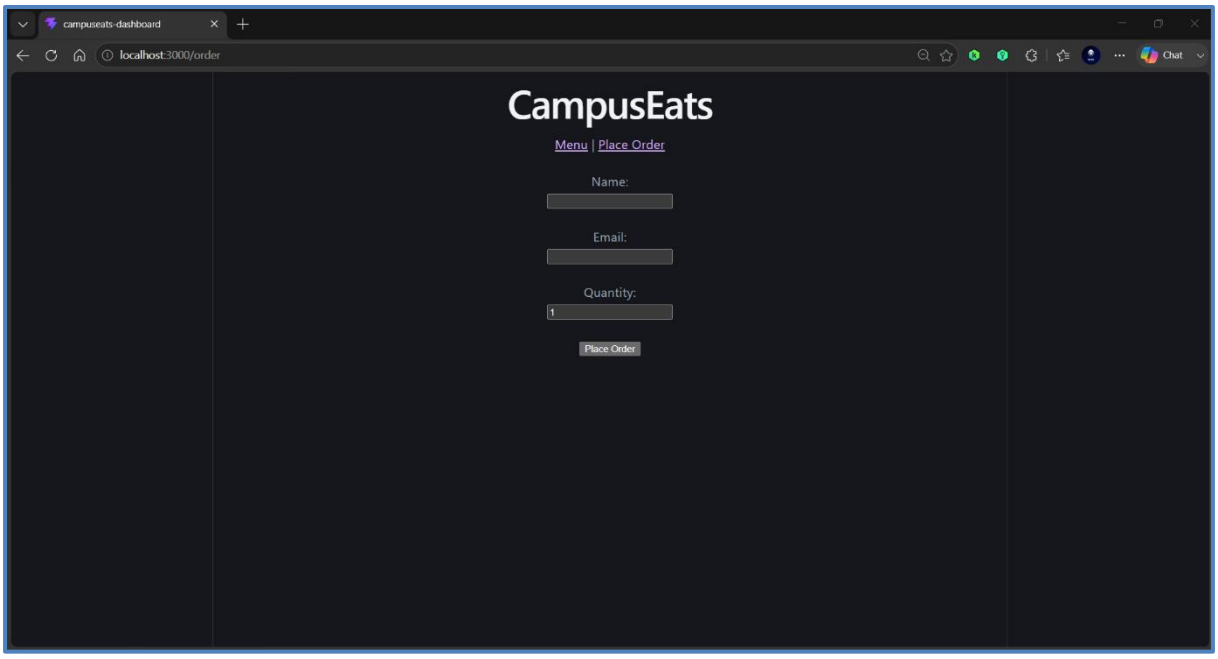


- **Screenshot of the Routing of Detailed Pages of Each Dish – 03**

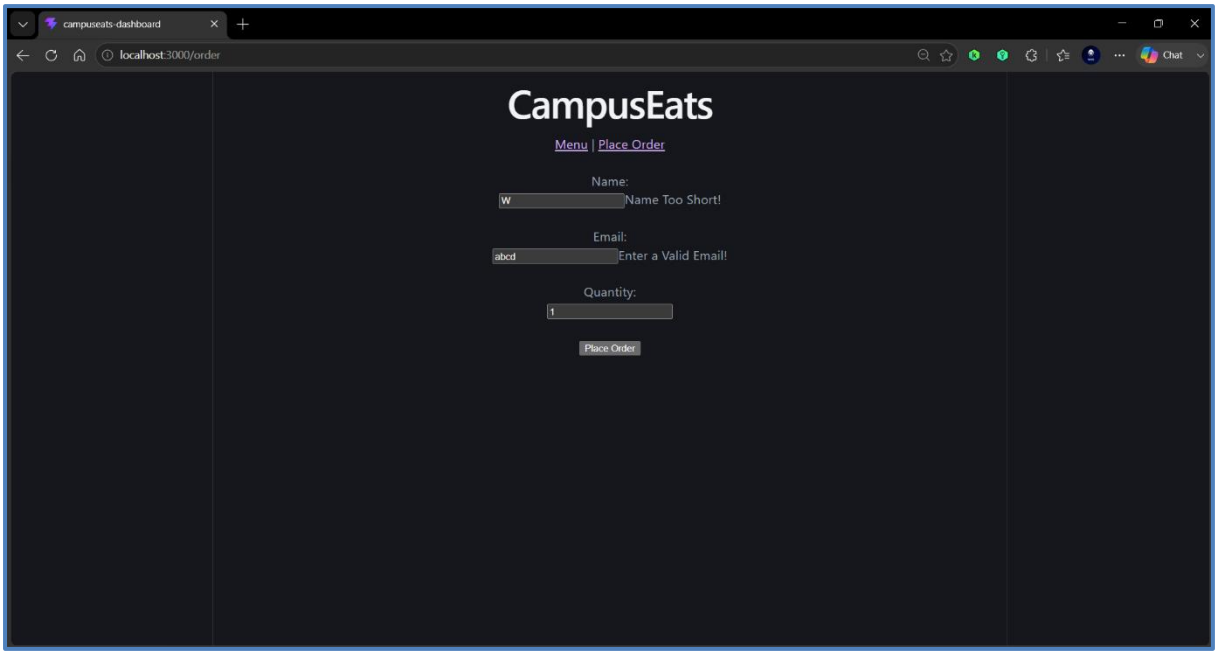


Part B – A Validated Order Form

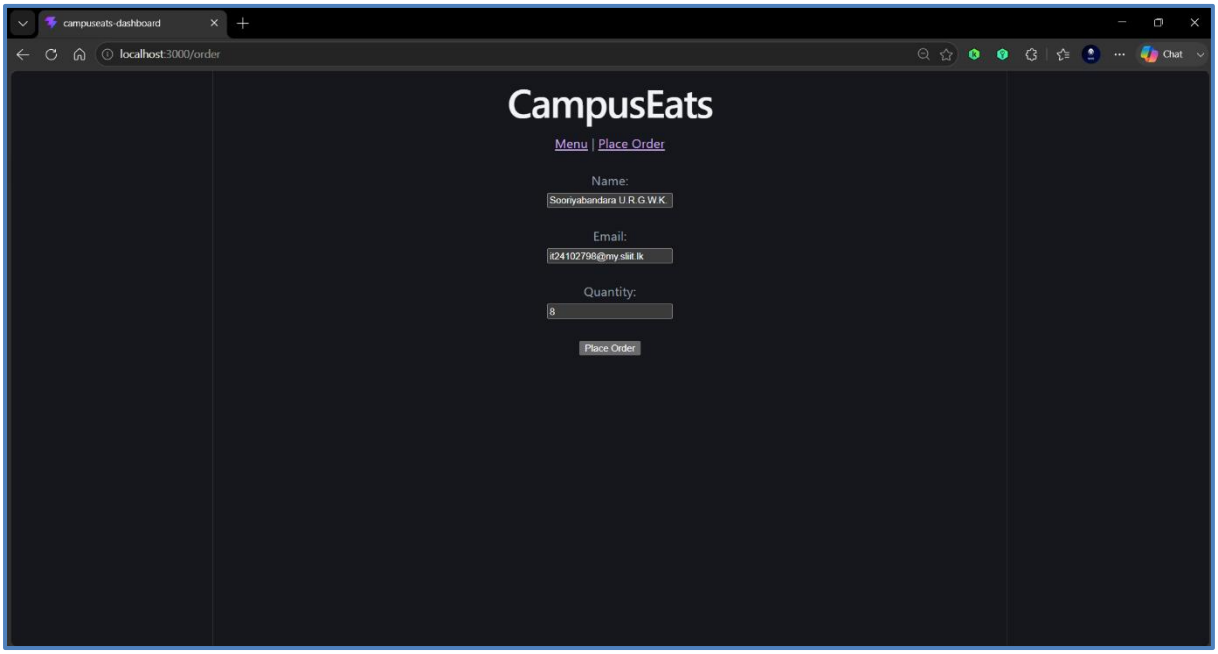
- **Screenshot of the Order Form**



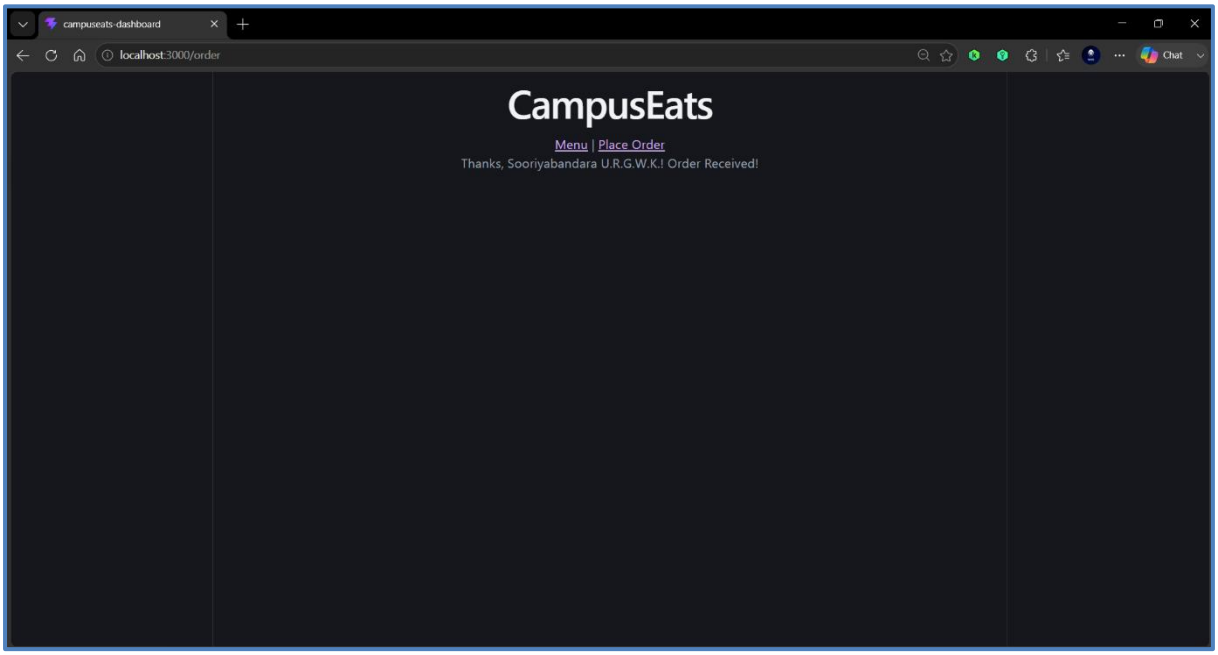
- **Screenshot of the Incompletely Filled Order Form**



- **Screenshot of the Completely Filled Order Form**



- **Screenshot of the Successfully Placed Order Form**

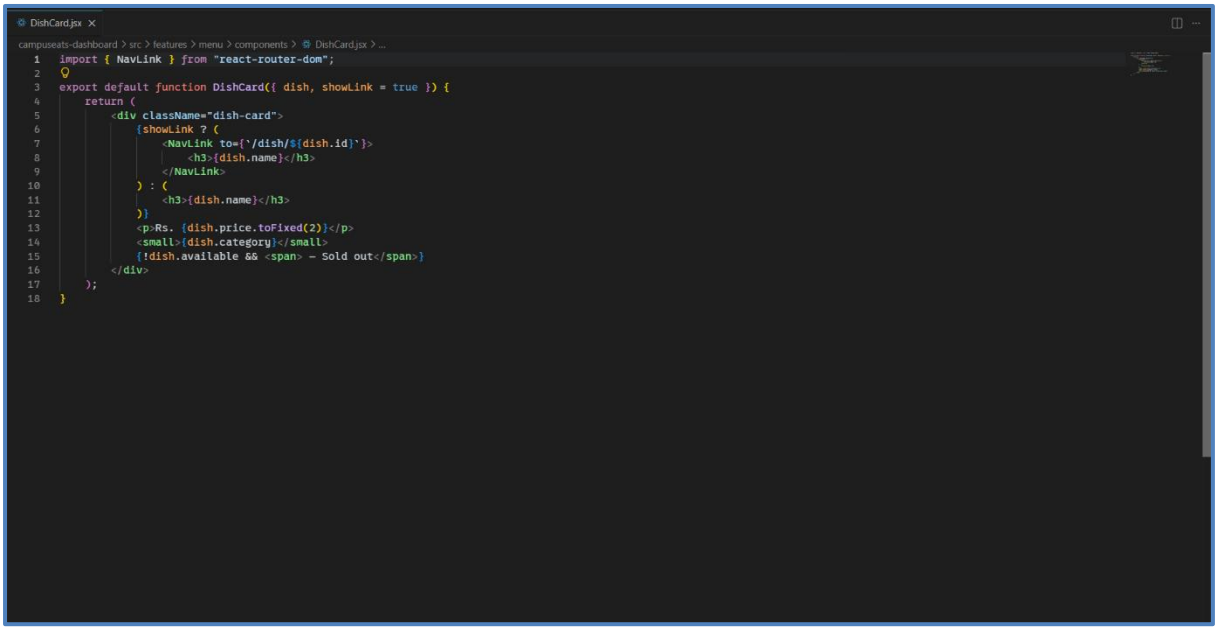


Part C – Justify your state choices (LO4)

Piece of State	Tool you would use	Why (One Line)
Search Box Text	useState	It is local UI state used only within the MenuPage component.
Menu Data from the API	useFetch	It manages asynchronous server data while providing loading, success, and error states.
Logged-in User (Shared App-wide)	Context API	It allows user information to be shared across multiple components without prop drilling.
A Cart of Selected Dishes	Context API	It stores shared cart data that needs to be accessed by multiple pages and components.

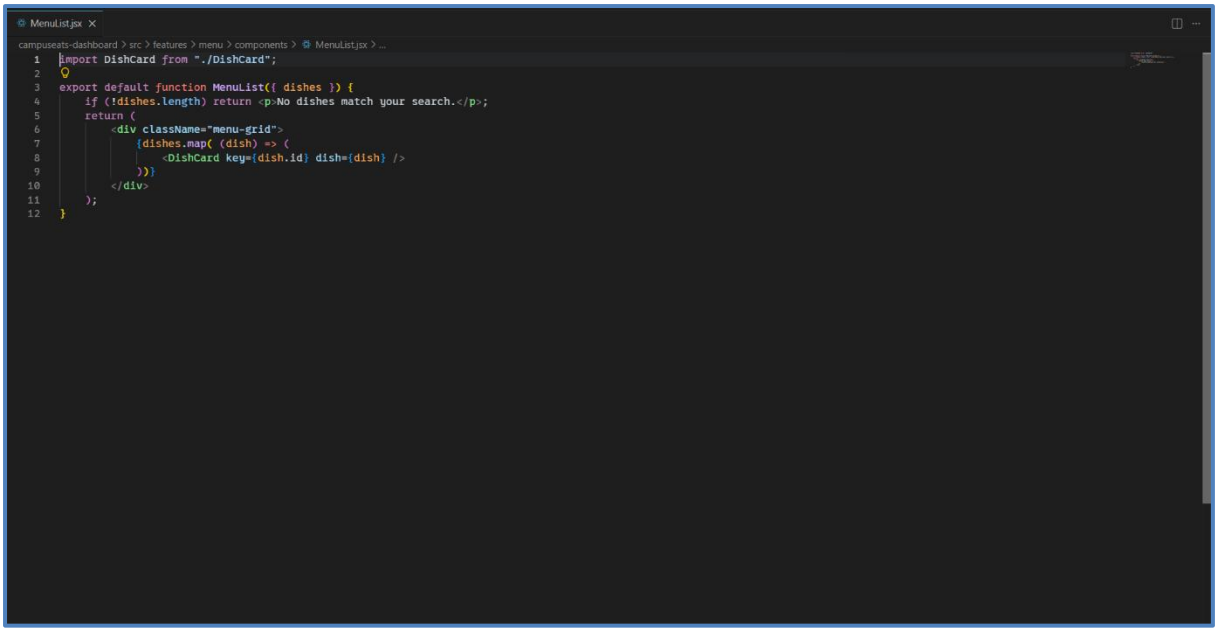
Extra – Source-code Snippets

- **DishCard.jsx**



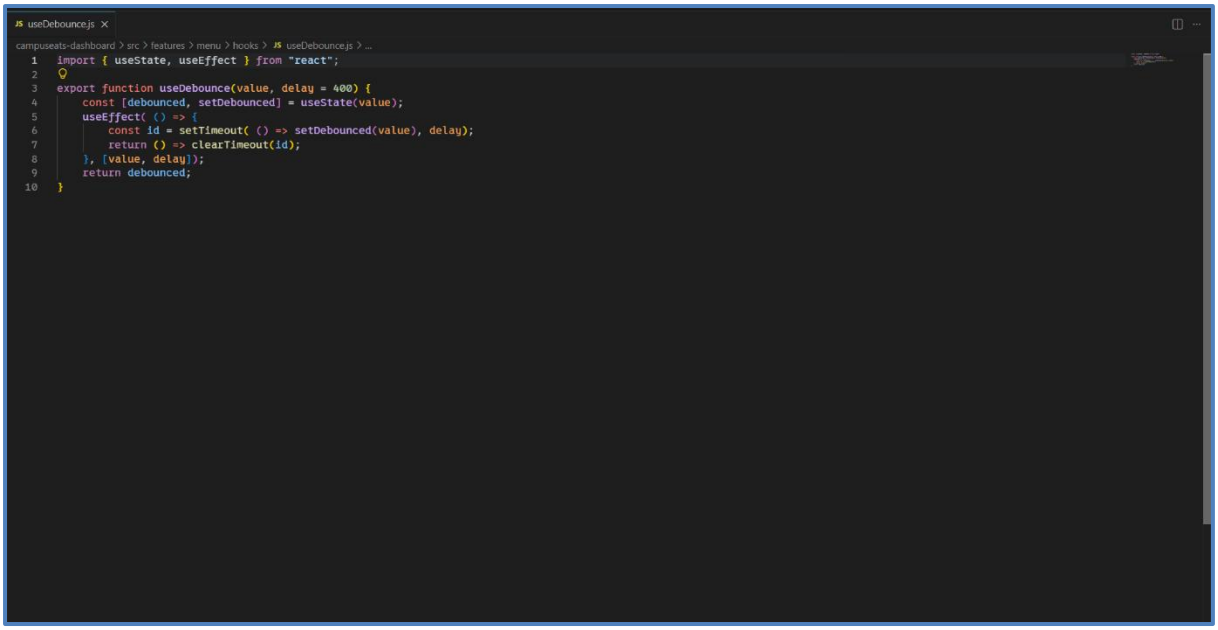
```jsx
import { NavLink } from "react-router-dom";

export default function DishCard({ dish, showLink = true }) {
    return (
        <div className="dish-card">
            {showLink ? (
                <NavLink to={`/dish/${dish.id}`}>
                    <h3>{dish.name}</h3>
                </NavLink>
            ) : (
                <h3>{dish.name}</h3>
            )}
            <p>Rs. {dish.price.toFixed(2)}</p>
            <small>{dish.category}</small>
            {!dish.available && <span> – Sold out</span>}
        </div>
    );
}
```

- **MenuList.jsx**

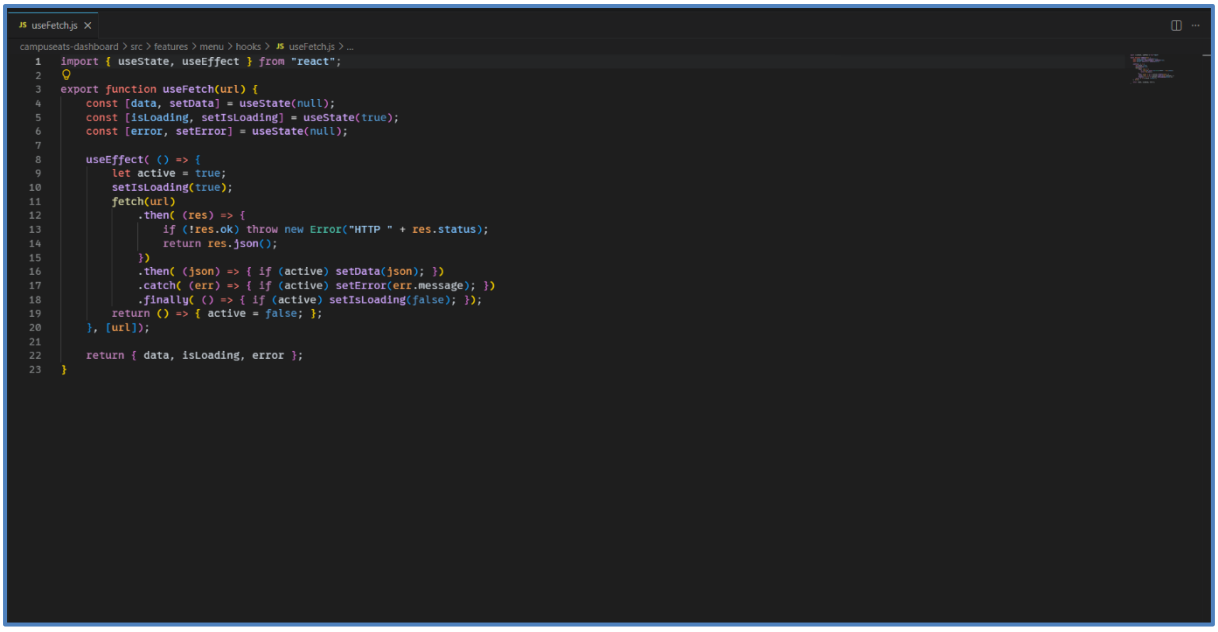


```jsx
import DishCard from "./DishCard";

export default function MenuList({ dishes }) {
    if (!dishes.length) return <p>No dishes match your search.</p>;
    return (
        <div className="menu-grid">
            {dishes.map( (dish) => (
                <DishCard key={dish.id} dish={dish} />
            ))}
        </div>
    );
}
```

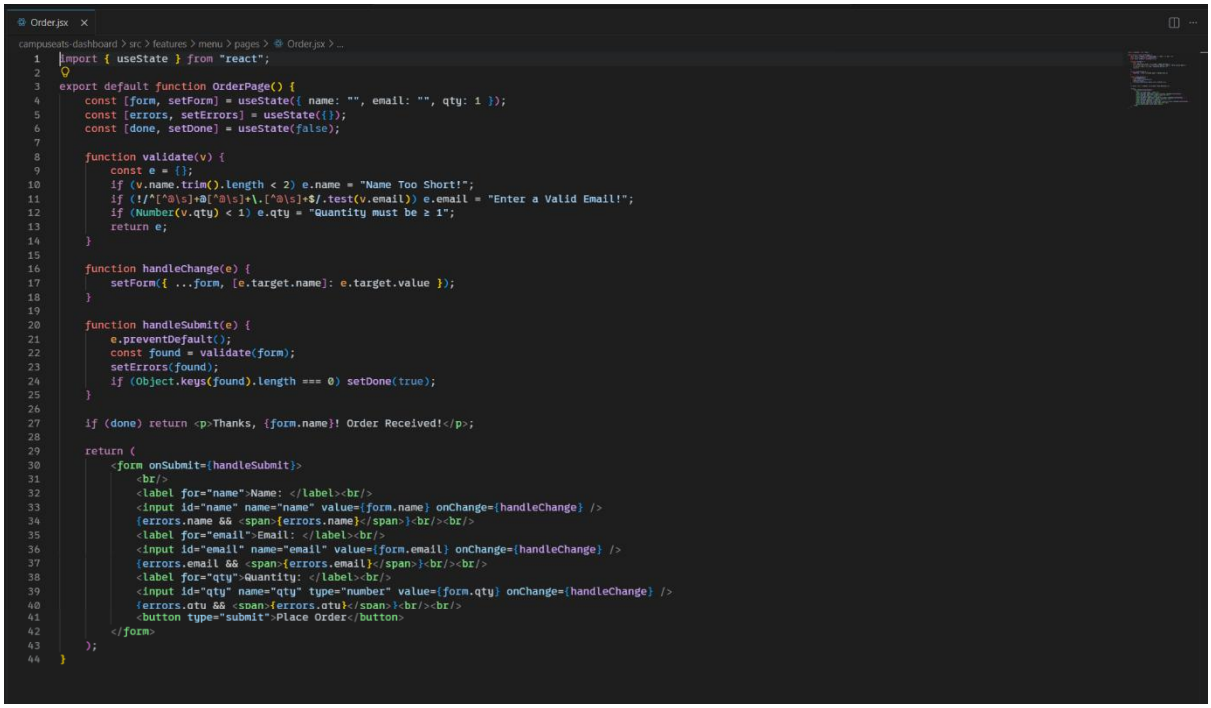
- **useDebounce.js**



- **useFetch,js**

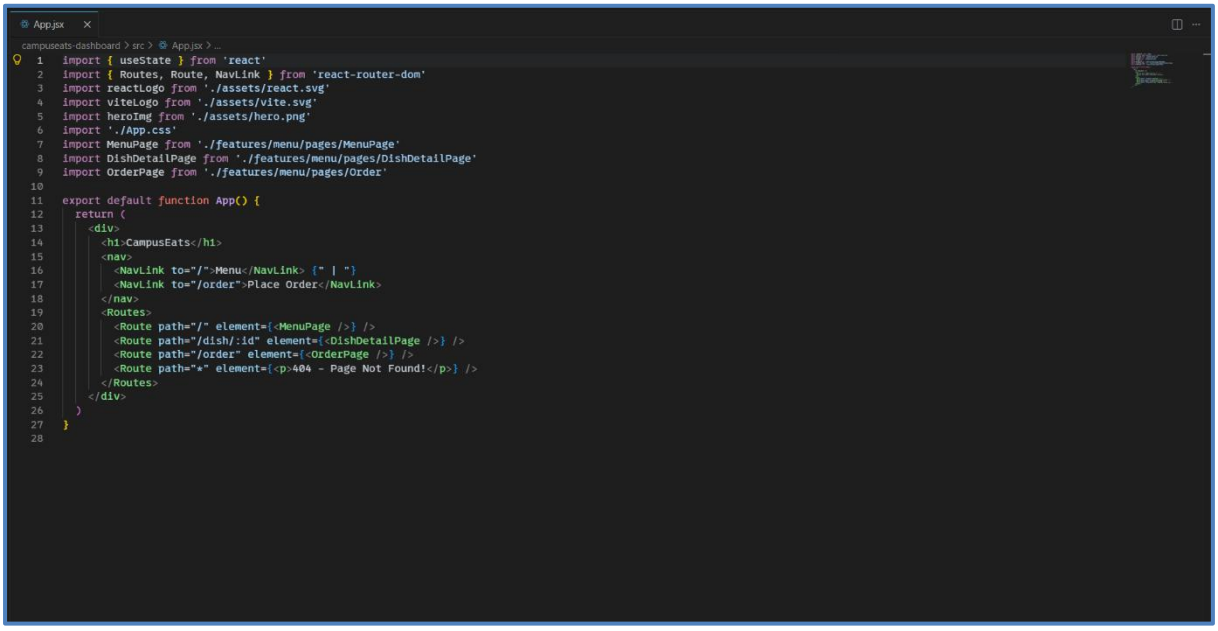


- **The Form (Order.jsx)**



```jsx
import { useState } from "react";

export default function OrderPage() {
    const [form, setForm] = useState({ name: "", email: "", qty: 1 });
    const [errors, setErrors] = useState({});
    const [done, setDone] = useState(false);

    function validate(v) {
        const e = {};
        if (v.name.trim().length < 2) e.name = "Name Too Short!";
        if (!/^[^@\s]+@[^@\s]+\.[^@\s]+$/.test(v.email)) e.email = "Enter a Valid Email!";
        if (Number(v.qty) < 1) e.qty = "Quantity must be ≥ 1";
        return e;
    }

    function handleChange(e) {
        setForm({ ...form, [e.target.name]: e.target.value });
    }

    function handleSubmit(e) {
        e.preventDefault();
        const found = validate(form);
        setErrors(found);
        if (Object.keys(found).length === 0) setDone(true);
    }

    if (done) return <p>Thanks, {form.name}! Order Received!</p>;

    return (
        <form onSubmit={handleSubmit}>
            <br/>
            <label for="name">Name: </label><br/>
            <input id="name" name="name" value={form.name} onChange={handleChange} />
            {errors.name && <span>{errors.name}</span>}<br/><br/>
            <label for="email">Email: </label><br/>
            <input id="email" name="email" value={form.email} onChange={handleChange} />
            {errors.email && <span>{errors.email}</span>}<br/><br/>
            <label for="qty">Quantity: </label><br/>
            <input id="qty" name="qty" type="number" value={form.qty} onChange={handleChange} />
            {errors.qty && <span>{errors.qty}</span>}<br/><br/>
            <button type="submit">Place Order</button>
        </form>
    );
}
```

- **The Routes (Inside App.jsx)**



```jsx
import { useState } from 'react'
import { Routes, Route, NavLink } from 'react-router-dom'
import reactLogo from './assets/react.svg'
import viteLogo from './assets/vite.svg'
import heroImg from './assets/hero.png'
import './App.css'
import MenuPage from './features/menu/pages/MenuPage'
import DishDetailPage from './features/menu/pages/DishDetailPage'
import OrderPage from './features/menu/pages/Order'

export default function App() {
  return (
    <div>
      <h1>CampusEats</h1>
      <nav>
        <NavLink to="/">Menu</NavLink> {" | "}
        <NavLink to="/order">Place Order</NavLink>
      </nav>
      <Routes>
        <Route path="/" element={<MenuPage />} />
        <Route path="/dish/:id" element={<DishDetailPage />} />
        <Route path="/order" element={<OrderPage />} />
        <Route path="*" element={<p>404 - Page Not Found!</p>} />
      </Routes>
    </div>
  )
}
```

Quick Test – Knowledge Check

Multiple Choice (Choose One)

Q1. (b) server state

Q2. (c) TanStack Query

Q3. (b) an unstable (newly created) prop

Q4. (c) useCallback

Q5. (b) only if the API enforces authorization

Short Answer

Q6.

- In a feature-based folder structure, files are grouped by feature or domain (such as menu, order, or user) instead of by file type. This avoids the file-type (layer-based) anti-pattern, where components, hooks, and pages for the same feature are scattered across different folders, making the project harder to maintain as it grows.

Code-based

Q7.

```
const [query, setQuery] = useState("");
<input value={query} onChange = { (e) => setQuery( e.target.value ) } />
```

Scenario-based

Q8.

(a) Whether a modal is open,
- **Tool:** useState
- **Reason:** It is local UI state used only by a single component.

(b) The product list returned by the API,
- **Tool:** TanStack Query / useFetch
- **Reason:** It is server state that requires fetching, caching, and loading/error handling.

(c) The logged-in user shared across the whole app,
- **Tool:** Context API
- **Reason:** It is shared application-wide state that multiple components need to access.